**Mathematisch-Naturwissenschaftliche Fakultät**

**Lernbasierte Computer Vision**

Masterarbeit

# ScholarCopilot with Fine-Grained Citation Retrieval

Eberhard Karls Universität Tübingen
Mathematisch-Naturwissenschaftliche Fakultät
Wilhelm-Schickard-Institut für Informatik
Lernbasierte Computer Vision
Niklas Munkes, niklas.munkes@student.uni-tuebingen.de, 2025

Bearbeitungszeitraum: 14.04.-14.10.2025

Gutachter: Prof. Dr. Andreas Geiger, Universität Tübingen
Zweitgutachter: Prof. Dr. Carsten Eickhoff, Universität Tübingen
Betreuer: Haoyu He, Universität Tübingen

# Selbstständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Masterarbeit selbständig und nur mit den angegebenen Hilfsmitteln angefertigt habe und dass alle Stellen, die dem Wortlaut oder dem Sinne nach anderen Werken entnommen sind, durch Angaben von Quellen als Entlehnung kenntlich gemacht worden sind. Diese Masterarbeit wurde in gleicher oder ähnlicher Form in keinem anderen Studiengang als Prüfungsleistung vorgelegt.

---

Niklas Munkes (Matrikelnummer 4269436), September 9, 2025

# Abstract

TODO

# Contents

# 1 Introduction

TODO

citations build trust [DOW+24]

Retrieval-Augmented Generation (RAG, [LPP+21]) models are designed to generate text that is grounded in real-world factual knowlege. They have been widely adopted for knowledge-intensive NLP tasks [SQK+25] such as long-form question answering [XWL+24], scientific question answering [LOS+23] and question synthesis [XLS+24], generation with sentence-level citations [ZBL+24, XWL+24] and academic text generation [AHS+24, WMN+25]. RAG models usually consist of a generator, a pre-trained sequence-to-sequence (seq2seq) model (e.g. a Transformer-based [VSP+17] decoder, see section 2), and a retriever with access to a retrieval corpus, which may be implemented as a dense vector index (e.g. a HNSW [MY16] index, see section 3.2.1) [LPP+21]. Thus they combine pre-trained paramentric memory with non-paramentric (retrieval-based) memory [LPP+21].

The RAG architecture [LPP+21] is comprised of a retriever $p_\eta$ consisting of a query encoder $q$ and a retrieval corpus $d$, and a generator $p_\theta$ (see Figure 1.1). Given a query $x$, the retriever $p_\eta(z|x)$ (with parameters $\eta$) computes a distribution over the text documents $z$ in the retrieval corpus $d$ according to $x$ [LPP+21]. This distribution is used to obtain the top-k most relevant documents with respect to the query. The generator $p_\theta(y_i|x, z, y_{1:i-1})$, parametrized by $\theta$, then predicts each next token $y_i$ based on the previous tokens $y_{1:i-1}$, the given query $x$ and a document $z$ from the top-k retrieved documents (according to $p_\eta$) [LPP+21]. Lewis et al. base their retriever on the `Dense Passage Retriever` [KOM+20] and use a pre-trained `BART-large` [LLG+19] as the generator.

While neural language models are able to store a substantial amount of world knowledge in their parameters [RRS20], this knowledge can't easily be modified, expanded or directly be used to gain insight into their decision-making process [LLG+19]. By using an external retrieval corpus, the knowledge RAG models use to inform their generation can directly be inspected and interpreted [LPP+21]. Their TODO

TODO

TODO

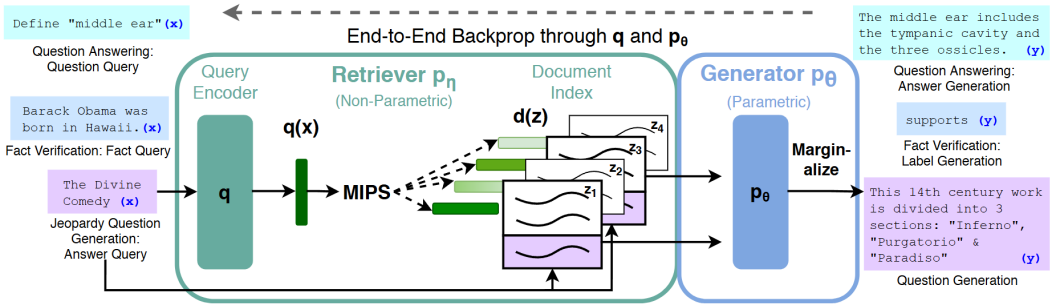


Figure 1.1: Overview of the RAG architecture initially proposed by [LPP+21]. For finding the top-k documents $Z'$, the authors used Maximum Inner Product Search (MIPS).

# 2 Related Work

TODO

## 2.1 Qwen2

TODO

## 2.2 Deepseek R1

[STB25] "Using human experts to evaluate the quality of LLM answers is generally considered the gold standard, but expert annotation is costly and slow" [HPS+25]
TODO

## 2.3 RAG for Scientific Writing

Previous work on text generation with citations has recently transitioned from treating retrieval and generation as seperate processing stages (e.g. [SHC+24]) to alternating between the two (e.g. [XWL+24], [AHS+24]). *Attribute First, then Generate* [SHC+24] initially retrieves a set of relevant sub-sentence spans which are then grouped into coherent clusters which in turn are used to inform an iterative sentence-by-sentence generation step [SHC+24]. *ReClaim* [XWL+24], more specifically the *ReClaim with Interleaving Generation* method, is used to generate an output sequence $O = \{r_1, c_1, r_2, c_2, ..., r_n, c_n\}$ of alternating references and claims where claim $c_i$ is the portion of the models response that was generated based on reference $r_i$. Claim and reference generation is performed by seperate LLMs specifically trained for their corresponding task [XWL+24]. In contrast, *OpenScholar* [AHS+24] first retrieves and reranks a set of papers $P$ relevant to the given query. These papers inform a subsequent generation step that produces a generated response $r_i$ [AHS+24]. This response is refined through iterative *self-feedback* where during each iteration of the feedback loop $F = f_1, f_2, ..., f_T$, the generator model generates sentences $f_j$ that describe potential improvements to the response and may also prompt the retrieval pipeline to retrieve additional papers, should $r_i$ require more information to generate an improved response $r_{i+1}$ [AHS+24]. Depending on the size of $P$ and the choice of $T$, OpenScholar still performs retrieval and generation largely seperately.

ScholarCopilot [WMN+25] is a RAG system developed for generating academic-style text with accurate citations. It is intended as a tool that assists the user with academic paper writing by providing iterative text generation with automated citation retrieval [WMN+25]. ScholarCopilot also enables user interaction during generation [WMN+25] by allowing the user to rewrite the generated text or forcefully trigger citation retrieval between the consecutive generation steps but this feature is beyond the scope of this thesis and is thus not utilized for the experiments. ...

# 3 Methods

TODO

## 3.1 Dataset

TODO [HFB+24]

## 3.2 ScholarCopilot

Instead of treating retrieval and generation as separate processing stages (see section 1) ScholarCopilot [WMN+25] integrates the information retrieval directly into the generation process. The model is trained to generate *retrieval tokens* (`[RET]`), that upon being generated, cause the model to interupt the generation process to retrieve a reference from the retrieval corpus [WMN+25]. The ScholarCopilot retrieval corpus[1] consists of abstracts of scientific papers, which are encoded by the ScholarCopilot model prior to training/inference and stored as a HNSW [MY16] index using Faiss [DGD+24] for efficient retrieval (see section 3.2.1). Standard ScholarCopilot uses the retrieved abstract to inform its subsequent generation by replacing the `[RET]` tokens in the generated context with the corresponding abstracts.

For this thesis, I evaluated the official ScholarCopilot implementation[2]. In their paper, the authors state that ScholarCopilot can also integrate key excerpts (in the paper it says "key excepts", likely a typo) [WMN+25] but the official implementation only uses abstracts and provides no logic for extracting these excerpts. Because of that, I further on use the term "ScholarCopilot" to denote a version of this model that represents papers by their abstracts and does not use potentially more informative key excerpts.

ScholarCopilot is intended to assist with academic writing by providing "[...] text completion and citation suggestions" (see the official demo[3,4] for an example). In the

[1] `https://huggingface.co/datasets/TIGER-Lab/ScholarCopilot-Data-v1`

[2] See `https://github.com/TIGER-AI-Lab/ScholarCopilot`. I cloned the repository on the 13th of June 2025 and evaluated that version, which corresponds to commit `f8e6de9` in the source repository. As of the time of writing this (7th of September), only the README.md was extended.

[3] `https://huggingface.co/spaces/TIGER-Lab/ScholarCopilot`

[4] `https://www.youtube.com/watch?v=QlY7S52sWDA`

following we will thus assume that the model is used to generate a scientific paper with citations. Let

$$Y^{\texttt{in}} = \left(y_1^{\texttt{in}}, y_2^{\texttt{in}}, ..., y_l^{\texttt{in}}\right) \tag{3.1}$$

denote the initial context given to the model. Since we seek to generate a scientific document, this context may consist of the title and abstract of the paper the model is supposed to generate. Let furthermore

$$Y_{1:i-1}^{\texttt{gen}} = \left(y_1^{\texttt{gen}}, y_2^{\texttt{gen}}, ..., y_{j-1}^{\texttt{gen}}, c_j, y_{j+1}^{\texttt{gen}}, ..., y_{i-1}^{\texttt{gen}}\right) \tag{3.2}$$

denote a token sequence generated by the model where $c_j$ signifies that a `[RET]` token was generated by the model at generation step $j$. Note that the model may generate the retrieval token at multiple steps, and that the number of generated retrieval tokens is equal to the number of citations in the generated paper. The representation $\hat{c}_j$ of a `[RET]` token $c_j$ computed by the model serves as an aggregated representation of the token sequence generated up to the `[RET]` token (e.g. $Y_{1:j-1}^{\texttt{gen}}$ in Eq. 3.2), in the same manner as BERT [DCLT19] (and also for example CF-ViT [CLL+22]) uses the `[cls]` token as a representation of its input sequence. Thus $\hat{c}_j$ is unique for every $j$ since it is dependent of the corresponding left-hand context.

When the ScholarCopilot generator $p_G$ generates a retrieval token (that is, $p_G(*|Y^{\texttt{in}}Y_{1:i-1}^{\texttt{gen}})$ is maximal for the `[RET]` token, thus $* \leftarrow c_i$), the generation process is interrupted and instead, the representation $\hat{c}_i$ is passed to the retriever $p_R(z_i|\hat{c}_i)$ used to retrieve an encoded reference $z_i$ corresponding to an abstract

$$Y_i^{\texttt{ref}} = \left(y_{i,1}^{\texttt{ref}}, y_{i,2}^{\texttt{ref}}, ..., y_{i,k}^{\texttt{ref}}\right) \tag{3.3}$$

from the retrieval corpus. Before passing the newly generated context to the generator the retrieval token $c_i$ is replaced with the corresponding abstract to obtain an augmented generation output

$$\mathbb{Y}_{1:i}^{\texttt{gen}} = \left(y_1^{\texttt{gen}}, y_2^{\texttt{gen}}, ..., y_{i-1}^{\texttt{gen}}, y_{i,1}^{\texttt{ref}}, y_{i,2}^{\texttt{ref}}, ..., y_{i,k}^{\texttt{ref}}\right) \tag{3.4}$$

that is used to inform the sebsequent generation steps with the retrieved information. The generator $p_G(y_{i+1}|Y^{\texttt{in}}\mathbb{Y}_{1:i}^{\texttt{gen}})$ predicts the next token $y_{i+1}$ at generation step $i+1$ based on the given initial context $Y^{\texttt{in}}$ and the augmented output $\mathbb{Y}_{1:i}^{\texttt{gen}}$. See Algo. 1 for a detailed overview of one generation step. Before presenting the generated sequence to the user, each inserted abstract is replaced with a token referencing the cited paper (when using latex formatting, this would for example be a citation like "\cite{vaswani2017attention}"). The user is presented with this modified sequence, as well as a list of all cited references.

### 3.2.1 HNSW

TODO

**Algorithm 1:** Overview of one generation step of ScholarCopilot. Here $Y$ denotes the vocabulary (the set of all tokens), $Z$ the retrieval corpus (all encoded abstracts), $G$ the ScholarCopilot backbone model (see section 2.1) and getAbstract() a function that returns the unencoded abstract corresponding to $z_i$

**Input:** The initial context $Y^{\texttt{in}}$ and the previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$
**Output:** The initial context $Y^{\texttt{in}}$ and the new generated sequence $Y^{\texttt{gen}}_{1:i}$

**Algorithm:** ScholarCopilotGenerationStep($Y^{\texttt{in}}, Y^{\texttt{gen}}_{1:i-1}$)
$y_i \leftarrow \arg\max_{y_i \in Y} p_G(y_i | Y^{\texttt{in}} Y^{\texttt{gen}}_{1:i-1})$
**if** $y_i$ = *[RET]* **then**
  $\hat{c}_i \leftarrow G(y_i)$
  $z_i \leftarrow \arg\max_{z_i \in Z} p_R(z_i | \hat{c}_i)$
  $Y^{\texttt{ref}}_i \leftarrow$ getAbstract($z_i$)
  $\mathbb{Y}^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} Y^{\texttt{ref}}_i$
  **return** $Y^{in}$, $\mathbb{Y}^{gen}_{1:i}$
**end**
**else**
  $Y^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} y_i$
  **return** $Y^{in}$, $Y^{gen}_{1:i}$
**end**

### 3.2.2 ScholarCopilot Training

TODO

## 3.3 Passage Reranking

As outlined in section 3.2, ScholarCopilot (SC) represents each scientific document in the retrieval corpus with its abstract. But, depending on the context for which the model is supposed to retrieve a reference, the abstract might not be the most informative passage of a candidate paper (see section 1). Thus always using a paper's abstract as reference may add irrelevant information into the generated sequence which can negatively affect the model's reasoning capabilities [SCM+23].

This thesis introduces a *Passage Retrieval* module (PR) that is supposed to enhance the citation accuracy and generation quality of ScholarCopilot (see section 1). The module is agnostic to the RAG model, which makes it possible to use with any RAG architecture. However, this thesis only investigates how it affects ScholarCopilot. First, the module takes the top-$k_{SC}$ documents

$$Z_i = \left(D_1, D_2, ..., D_{k_{SC}}\right) \tag{3.5}$$

returned by the SC retriever for a previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$ and subdivides these documents into $n$-sized passages to obtain a set of candidate passages $P_i$. To preserve the semantic structure of each document $D_i$ in the passages obtained from it as much as possible, each document $D_i$ is first split into sections

$$D_i = \left(\mathcal{E}_1, \mathcal{E}_2, ..., \mathcal{E}_{|D|}\right) \tag{3.6}$$

by utilizing the fact that the HDT dataset [HFB+24] (see also section 3.1) already provides its documents structured into sections. Each section $\mathcal{E}_i$ is split into passages

$$\mathcal{E}_i = \left(\mathcal{P}_1, \mathcal{P}_2, ..., \mathcal{P}_{|\mathcal{E}_i|}\right) \tag{3.7}$$

which are sequences of $n$ or less tokens $\mathcal{P}_i = (t_1, t_2, ..., t_n)$ ($\mathcal{P}_{|\mathcal{E}_i|}$ might be shorter than $n$ tokens since $|\mathcal{E}_i| \equiv 0 \mod n$ likely does not hold for every section in every document). All passages are collected into a corpus $P_i$ and then ranked by a decoder model $R_{PR}$ based on how relevant they are to the context $Y^{\texttt{gen}}_{1:i-1}$ (assuming $y_i$ denotes the `[RET]` token that triggered the retrieval, see section 3.2, particularly Algo. 1). To reduce the computational burden, the context is truncated to size $w$ – yielding $Y^{\texttt{gen}}_{i-w-1:i-1}$ as the new context – before being passed to $R_{PR}$. I found that passing longform context tends to worsen the performance of the passage ranking model. The top-ranked passage is then integrated into the generated text in the same manner as the abstract would have been integrated when using standard ScholarCopilot (see Eq. 3.4).

How the PR module is integrated into ScholarCopilot is shown in Algo. 2. ...

### 3.3.1 Batched Self-Consistency

TODO

Self-consistency improves the performance of prompting techniques [SCM+23]

### 3.3.2 Min-Max Normalization

TODO

**Algorithm 2:** Overview of one generation step of ScholarCopilot with Passage Retrieval.

**Input:** The initial context $Y^{\texttt{in}}$ and the previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$
**Output:** The initial context $Y^{\texttt{in}}$ and the new generated sequence $Y^{\texttt{gen}}_{1:i}$

**Algorithm:** ScholarCopilotWithPRGenerationStep($Y^{\texttt{in}}, Y^{\texttt{gen}}_{1:i-1}$)
$y_i \leftarrow \arg\max_{y_i \in Y} p_G(y_i | Y^{\texttt{in}} Y^{\texttt{gen}}_{1:i-1})$
**if** $y_i$ = *[RET]* **then**
  $\hat{c}_i \leftarrow G(y_i)$
  $Z_i \leftarrow \emptyset$
  $Z' \leftarrow Z$
  **while** $|Z_i| < k_{SC}$ **do**
    $z_j \leftarrow \arg\max_{z_j \in Z'} p_R(z_j | \hat{c}_i)$
    $Z_i \leftarrow Z_i \cup \{z_j\}$
    $Z' \leftarrow Z' \setminus \{z_j\}$
  **end**
  $P_i \leftarrow \emptyset$
  **foreach** $z_j \in Z_i$ **do**
    $D_j \leftarrow$ getDocument($z_j$)
    **foreach** $\mathcal{P}_k \in D_j$ **do**
      $P_i \leftarrow P_i \cup \mathcal{P}_k$ // see Eq. 3.6 and Eq. 3.7
    **end**
  **end**
  $Y^{\texttt{ref}}_i \leftarrow \arg\max_{\mathcal{P}_k \in P_i} p_{R_{PR}}\left(\mathcal{P}_k | Y^{\texttt{gen}}_{i-w-1:i-1}\right)$
  $\mathbb{Y}^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} Y^{\texttt{ref}}_i$
  **return** $Y^{in}, \mathbb{Y}^{gen}_{1:i}$
**end**
**else**
  $Y^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} y_i$
  **return** $Y^{in}, Y^{gen}_{1:i}$
**end**

# 4 Experiments

TODO

## 4.1 Retrieval Evaluation

TODO

## 4.2 Generation Evaluation

TODO

## 4.3 Additional Experiments

TODO

# 5 Limitations and Future Work

TODO

# 6 Conclusion

TODO

# Bibliography


[AHS+24] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'arcy, David Wadden, Matt Latzke, Minyang Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Dan Weld, Doug Downey, Wen tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. Openscholar: Synthesizing scientific literature with retrieval-augmented lms, 2024.

[CLL+22] Mengzhao Chen, Mingbao Lin, Ke Li, Yunhang Shen, Yongjian Wu, Fei Chao, and Rongrong Ji. Cf-vit: A general coarse-to-fine method for vision transformer, 2022.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[DGD+24] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.

[DOW+24] Hyo Jin Do, Rachel Ostrand, Justin D. Weisz, Casey Dugan, Prasanna Sattigeri, Dennis Wei, Keerthiram Murugesan, and Werner Geyer. Facilitating human-llm collaboration through factuality scores and source attributions, 2024.

[HFB+24] Haoyu He, Markus Flicke, Jan Buchman, Iryna Gurevych, and Andreas Geiger. HDT: Hierarchical Document Transformer. Conference on Language Modeling, 2024.

[HPS+25] Sebastian Heineking, Jonas Probst, Daniel Steinbach, Martin Potthast, and Harrisen Scells. Ranking generated answers: On the agreement of retrieval models with humans on consumer health questions, 2025.

[KOM+20] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering, 2020.

[LLG+19] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer.

Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.

[LOS+23] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodriques, and Andrew D. White. Paperqa: Retrieval-augmented generative agent for scientific research, 2023.

[LPP+21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

[MY16] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, 2016.

[RRS20] Adam Roberts, Colin Raffel, and Noam Shazeer. How much knowledge can you pack into the parameters of a language model?, 2020.

[SCM+23] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context, 2023.

[SHC+24] Aviv Slobodkin, Eran Hirsch, Arie Cattan, Tal Schuster, and Ido Dagan. Attribute first, then generate: Locally-attributable grounded text generation, 2024.

[SQK+25] Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen tau Yih, Pang Wei Koh, and Luke Zettlemoyer. Reasonir: Training retrievers for reasoning tasks, 2025.

[STB25] Shubham Sharma, Sneha Tuli, and Narendra Badam. Challenges and applications of large language models: A comparison of gpt and deepseek family of models, 2025.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.

[WMN+25] Yubo Wang, Xueguang Ma, Ping Nie, Huaye Zeng, Zhiheng Lyu, Yuxuan Zhang, Benjamin Schneider, Yi Lu, Xiang Yue, and Wenhu Chen. ScholarCopilot: Training large language models for academic writing with accurate citations, 2025.

[XLS+24] Fangyuan Xu, Kyle Lo, Luca Soldaini, Bailey Kuehl, Eunsol Choi, and David Wadden. Kiwi: A dataset of knowledge-intensive writing instructions for answering research questions, 2024.

[XWL+24] Sirui Xia, Xintao Wang, Jiaqing Liang, Yifei Zhang, Weikang Zhou, Jiaji Deng, Fei Yu, and Yanghua Xiao. Ground every sentence: Improving retrieval-augmented llms with interleaved reference-claim generation, 2024.

[ZBL+24] Jiajie Zhang, Yushi Bai, Xin Lv, Wanjun Gu, Danqing Liu, Minhao Zou, Shulin Cao, Lei Hou, Yuxiao Dong, Ling Feng, and Juanzi Li. Longcite: Enabling llms to generate fine-grained citations in long-context qa, 2024.